TP n°4 :

Ex1 :

Exercice 1 — Calculer X^N par multiplications successives

Remarques : on suppose X et N entiers naturels (N ≥ 0). Pour éviter les erreurs d'affichage sur très grands exposants, j'utilise `unsigned long long` et j'avertis sur risque d'overflow.

```c
#include <stdio.h>

int main(void) {
    unsigned long long X;
    unsigned int N;
    unsigned long long result = 1;
    unsigned int i;

    printf("Entrez X (entier naturel) : ");
    if (scanf("%llu", &X) != 1) return 1;
    printf("Entrez N (entier naturel) : ");
    if (scanf("%u", &N) != 1) return 1;

    /* Multiplications successives */
    for (i = 0; i < N; ++i) {
        result *= X;
    }
```

```c
    printf("%llu^%u = %llu\n", X, N, result);
    return 0;
}
```

Exemple :

```cpp
Entrez X (entier naturel) : 3
Entrez N (entier naturel) : 4
3^4 = 81
```

EX2 :

Principe : l'ordinateur choisit un nombre entre MIN et MAX (ici 1..100), et l'utilisateur devine. On compte le nombre de coups.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    const int MIN = 1;
    const int MAX = 100;
    int nombreMystere, essai, coups = 0;

    /* Initialisation du générateur aléatoire */
    srand((unsigned int)time(NULL));
    nombreMystere = (rand() % (MAX - MIN + 1)) + MIN;

    printf("Devinez le nombre entre %d et %d\n", MIN, MAX);
```

```c
    do {
        printf("Votre proposition : ");
        if (scanf("%d", &essai) != 1) return 1;
        ++coups;
        if (essai < nombreMystere) {
            printf("C'est plus.\n");
        } else if (essai > nombreMystere) {
            printf("C'est moins.\n");
        } else {
            printf("Bravo ! Vous avez trouvé en %d coup(s).\n", coups);
        }
    } while (essai != nombreMystere);

    return 0;
}
```

Exemple d'interaction :

```python-repl
Devinez le nombre entre 1 et 100
Votre proposition : 50
C'est plus.
Votre proposition : 75
C'est moins.
...
Bravo ! Vous avez trouvé en 7 coup(s).
```

Ex3 :

Exercice 3 — Calculer la valeur d'un polynôme (schéma de Horner)

Principe (Horner) : si $P(X) = A_n X^n + A_{n-1} X^{n-1} + \cdots + A_0$,
on calcule `res = A_n; for i from n-1 downto 0: res = res * X + A_i`.

J'utilise `double` pour X et les coefficients.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n, i;
    double x, res;

    printf("Degré n (entier >= 0) : ");
    if (scanf("%d", &n) != 1 || n < 0) return 1;

    double *A = (double*)malloc((n+1) * sizeof(double));
    if (!A) return 1;

    /* On lit les coefficients A_n ... A_0 ou A_0 ... A_n.
       Ici on lit A_n à A_0 pour être direct avec Horner. */
```

```c
    printf("Entrez les coefficients du polynôme de A_n à A_0 (separés par espaces/newline):\n");
    for (i = n; i >= 0; --i) {
        printf("A_%d : ", i);
        if (scanf("%lf", &A[i]) != 1) { free(A); return 1; }
    }

    printf("Entrez X (float) : ");
    if (scanf("%lf", &x) != 1) { free(A); return 1; }

    /* Horner */
    res = A[n];
    for (i = n - 1; i >= 0; --i) {
        res = res * x + A[i];
    }

    printf("P(%.6g) = %.10g\n", x, res);
```

```c
    free(A);
    return 0;
}
```

```
Degré n (entier >= 0) : 3
Entrez les coefficients du polynôme de A_n à A_0 (separés par espaces/newline):
A_3 : 1
A_2 : 0
A_1 : -2
A_0 : 1
Entrez X (float) : 2
P(2) = 1
```

Ex4 :

Exercice 4 — Afficher un triangle isocèle d'étoiles sur N lignes

Principe : pour la i-ème ligne (i = 1..N) : afficher `N - i` espaces puis `2*i - 1` étoiles.

```c
#include <stdio.h>

int main(void) {
    int N, i, j;

    printf("Nombre de lignes : ");
    if (scanf("%d", &N) != 1 || N <= 0) {
        printf("N doit être un entier positif.\n");
        return 1;
    }
```

```c
    for (i = 1; i <= N; ++i) {
        /* espaces */
        for (j = 0; j < N - i; ++j) putchar(' ');
        /* étoiles */
        for (j = 0; j < 2 * i - 1; ++j) putchar('*');
        putchar('\n');
    }

    return 0;
}
```

Exemple (N = 8) produit exactement l'illustration de l'énoncé :

```markdown
        *
       ***
      *****
     *******
    *********
   ***********
  *************
***************
```